Trees
Data Structure Algorithm

Ashim Khadka

Master Program

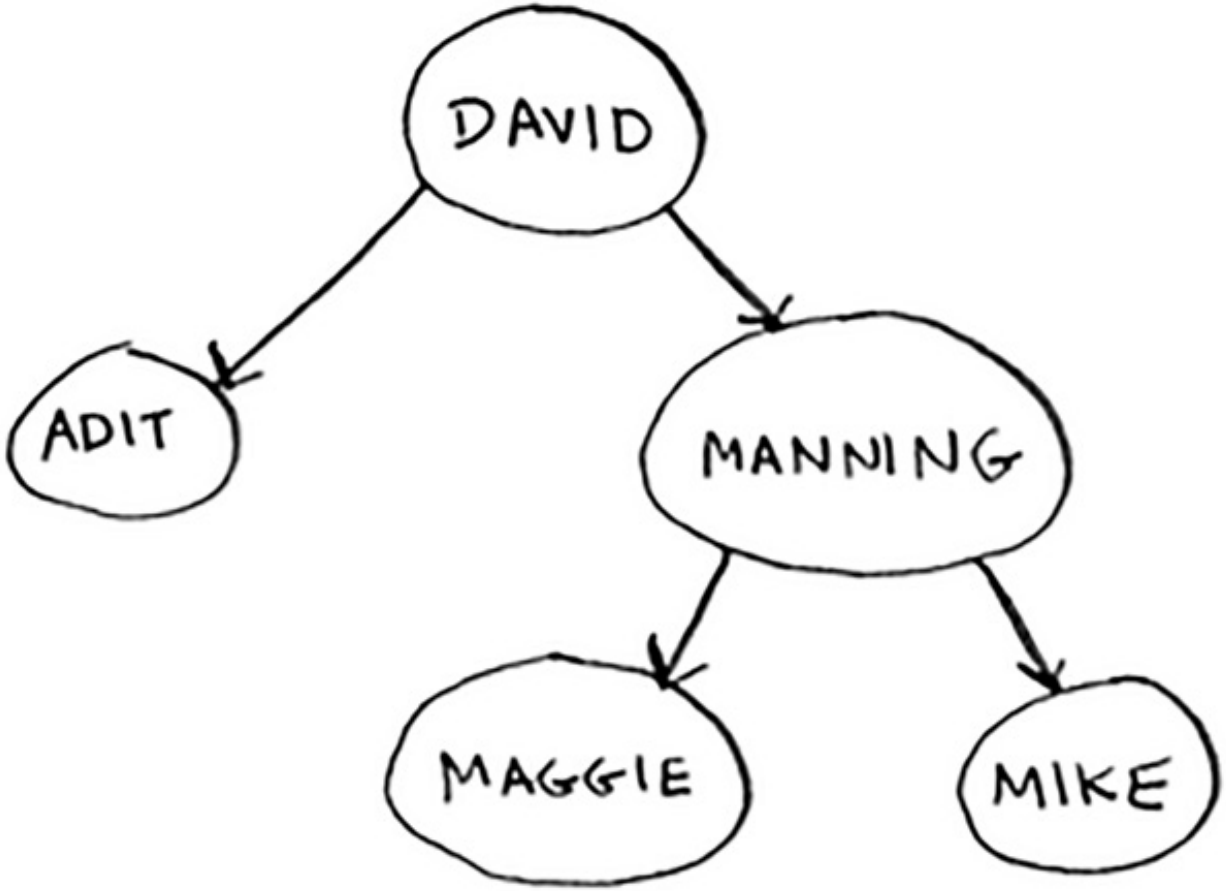
Outline

1. Binary Search Tree
2. Red-Black Tree
3. Huffman Coding
4. B-Tree

Tree

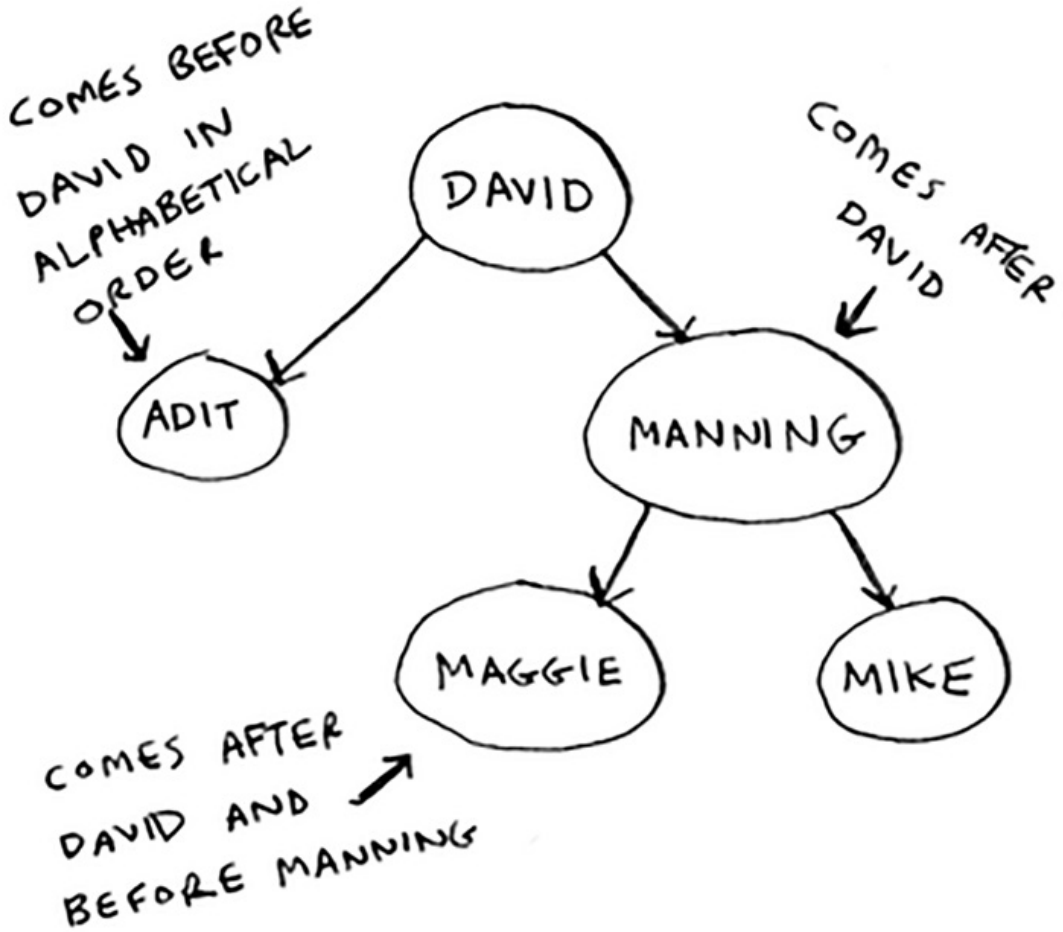
- user logs in to Facebook
- Facebook: look through a big array to see if the username exists
- Fastest way to search: binary search
- Problem: every time a new user signs up
  - insert their username into the array
  - re-sort the array, because binary search only works with **sorted arrays**

Binary Search Tree



Binary Search Tree

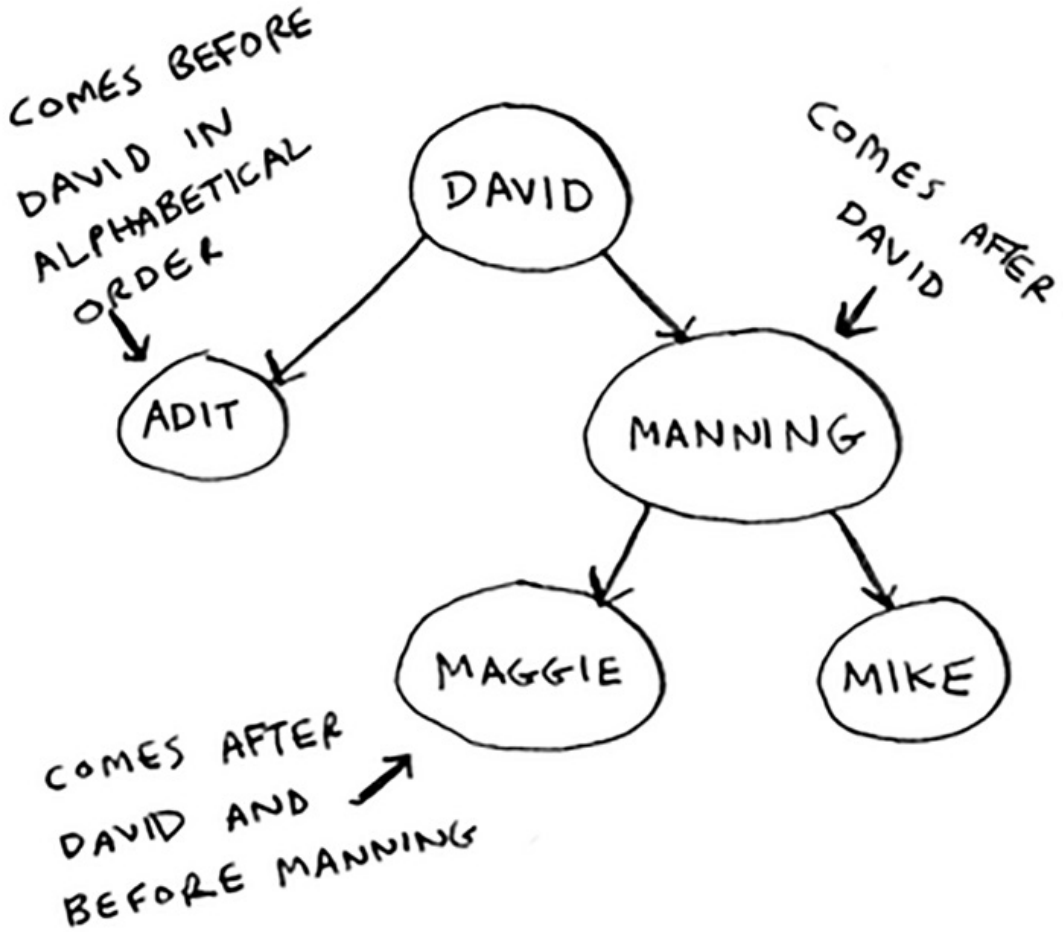
For every node, the nodes to its left are **smaller** in value, and the nodes to the right are **larger** in value



Time complexity: ??? (search, insert, delete)

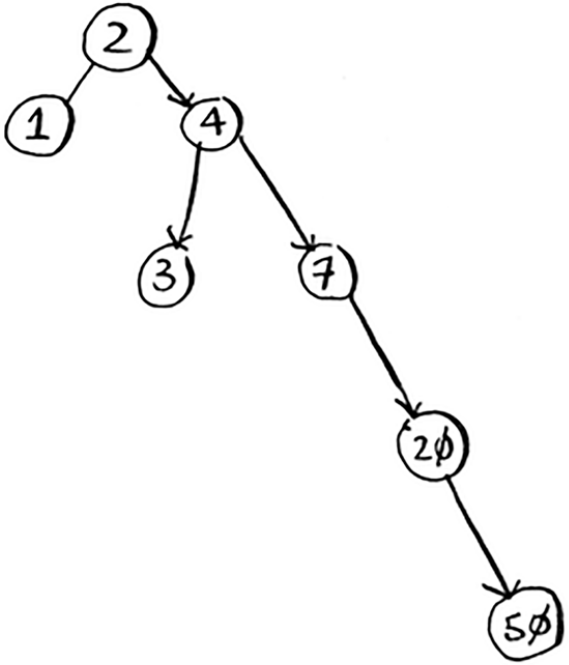
Binary Search Tree

For every node, the nodes to its left are **smaller** in value, and the nodes to the right are **larger** in value



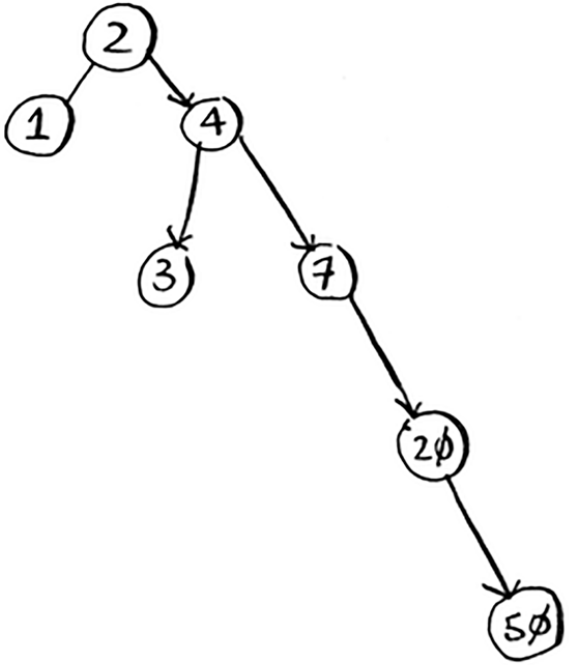
Time complexity: ??? (search, insert, delete) $O(\log n)$

Binary Search Tree



Not balanced
arr:[2,4,7,20,50]
Time complexity: ??? (search, insert, delete)

Binary Search Tree



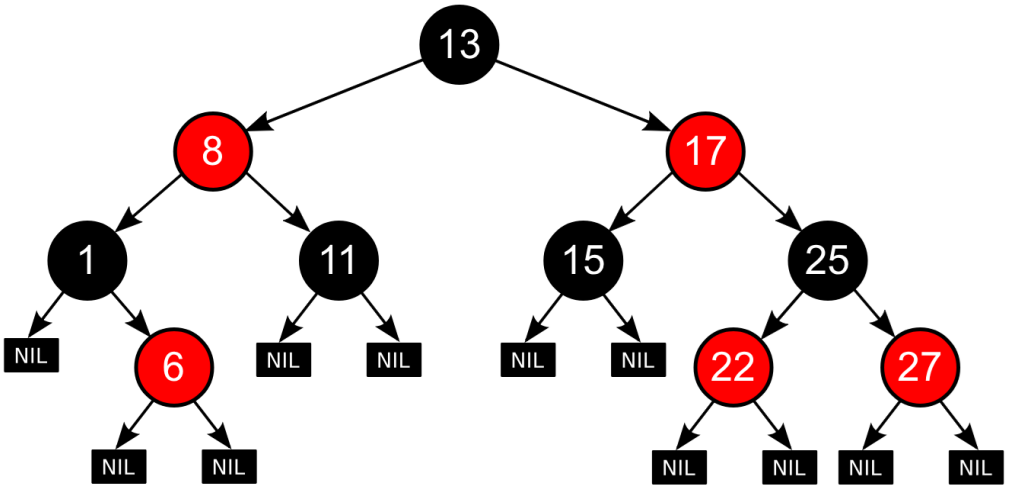
Not balanced
arr:[2,4,7,20,50]
Time complexity: ??? (search, insert, delete)
$O(n)$

Red-Black Tree

- binary search tree with one extra bit of storage per node
  - indicate its color: either **red** or **black**
- By constraining how nodes are colored on any simple path from the root to a leaf
  - trees ensure the structure remains approximately balanced
  - guarantee that no such path is more than twice as long as any other
- balancing property ensures that basic dynamic-set operations
  - search, insertion, and deletion-take $O(\log n)$ time in the absolute worst case

Properties of Red-Black Trees

1. Every node is either red or black
2. The root is black
3. Every leaf (NIL) is black
4. If a node is red, then both its children are black (meaning no two red nodes can be adjacent)
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes

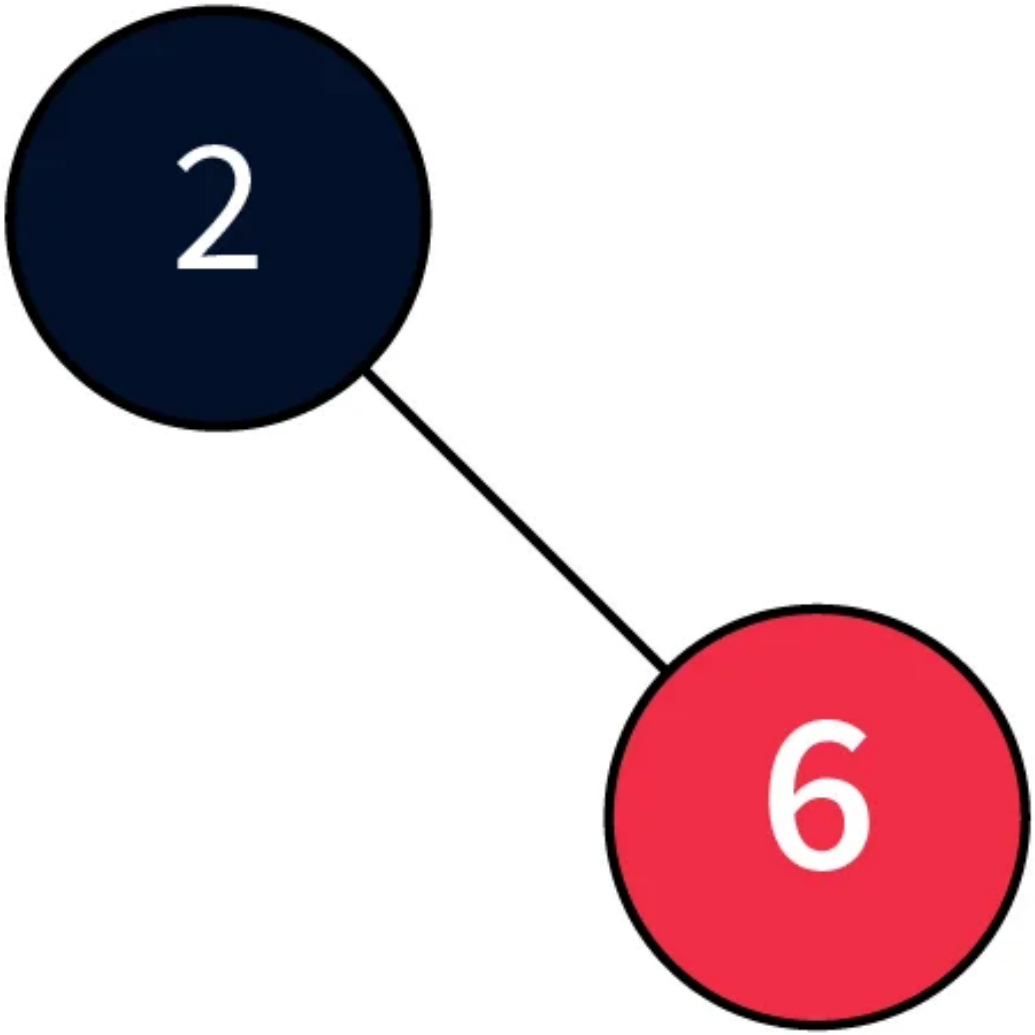


Insertion

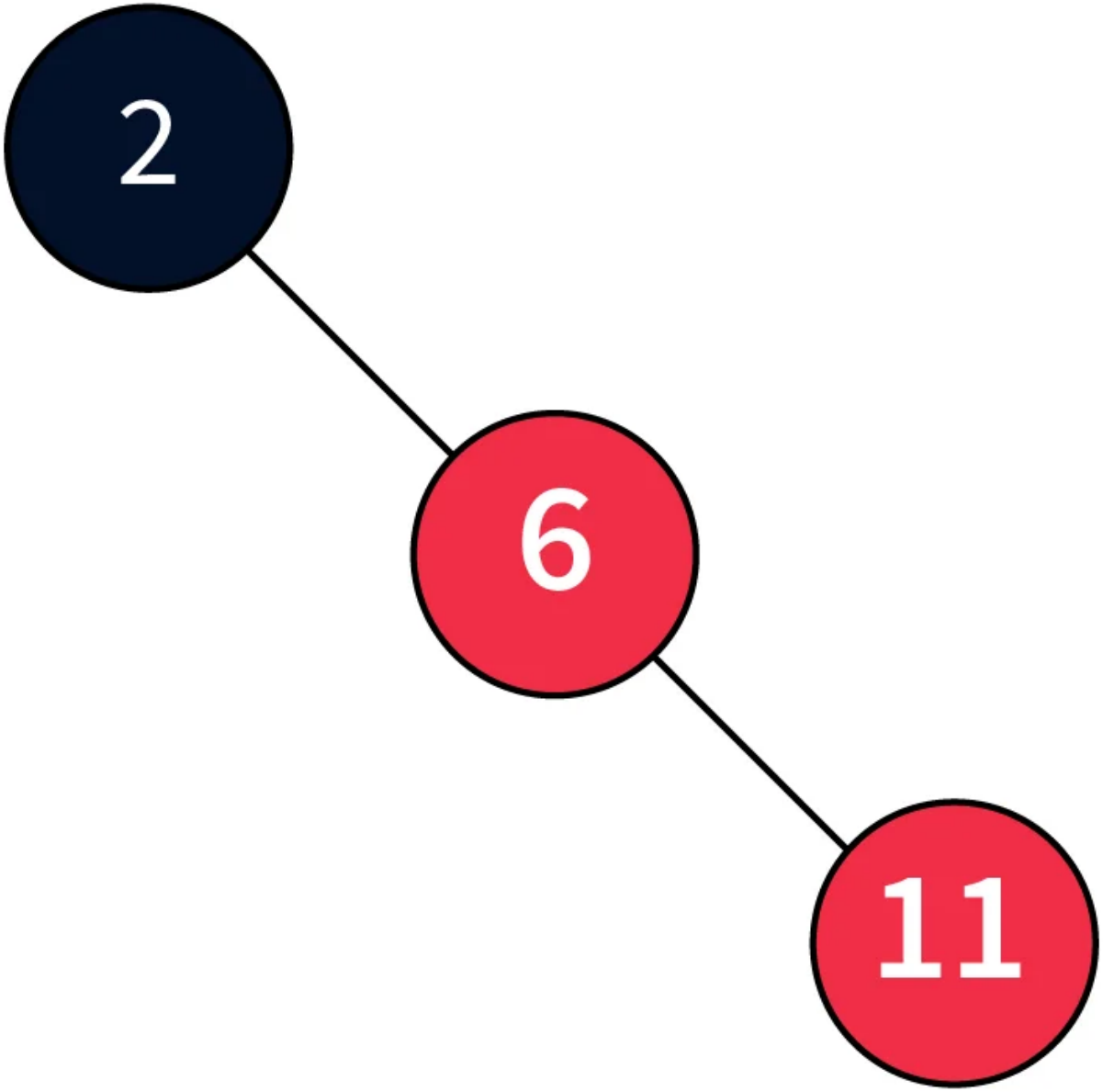
1. Insert the new node as BST with color red
2. Node is Root: recolor it to Black
3. Red-Red Violation
   - 3.1 Uncle is Red
     - Recolor parent and uncle to black, grandparent to red
     - Move focus to the grandparent and repeat checks
   - 3.2 Uncle is Black (or Null): Triangle Shape (Zig-Zag)
     - 3.2.1 Uncle is Black and Node is a Left child of parent
       - a Perform a right rotation on the Parent to transform the tree into a "Line" shape
       - b Converts to Case 3.2.3
     - 3.2.2 Uncle is Black and Node is a right child of parent
       - a Perform a left rotation on the Parent to transform the tree into a "Line" shape
       - b Converts to Case 3.2.3
     - 3.2.3 Uncle is Black: Line Shape
       - a If the node is a left child: Perform a Right Rotation on Grandparent
       - b If the node is a right child: Perform a Left Rotation on Grandparent
       - c Recolor: Swap the colors of the original Parent and the Grandparent
       - d Restores Red-Black properties

2, 6, 11, 17, 22, 30, 49, 58

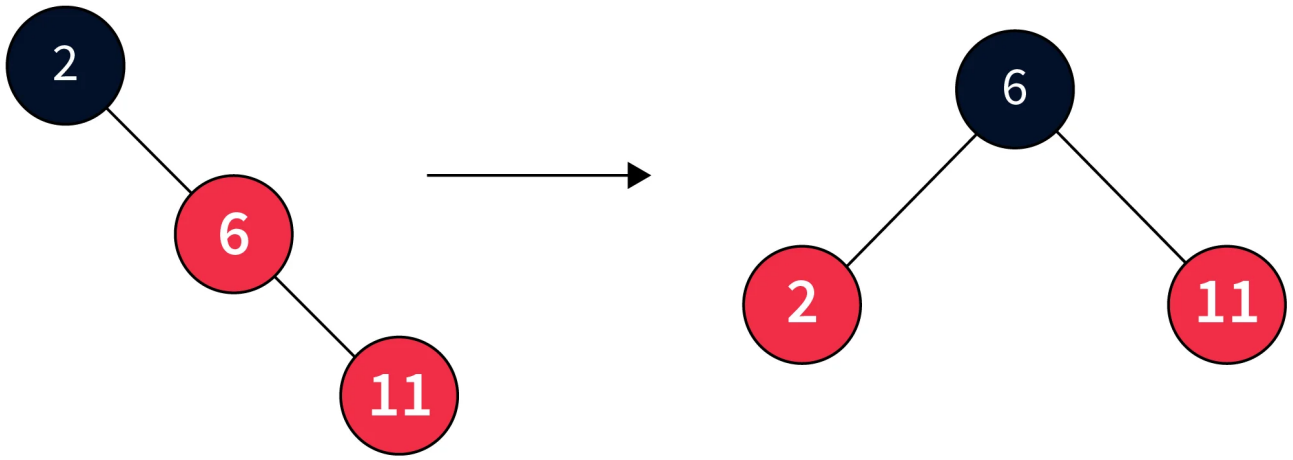
2, 6, 11, 17, 22, 30, 49, 58



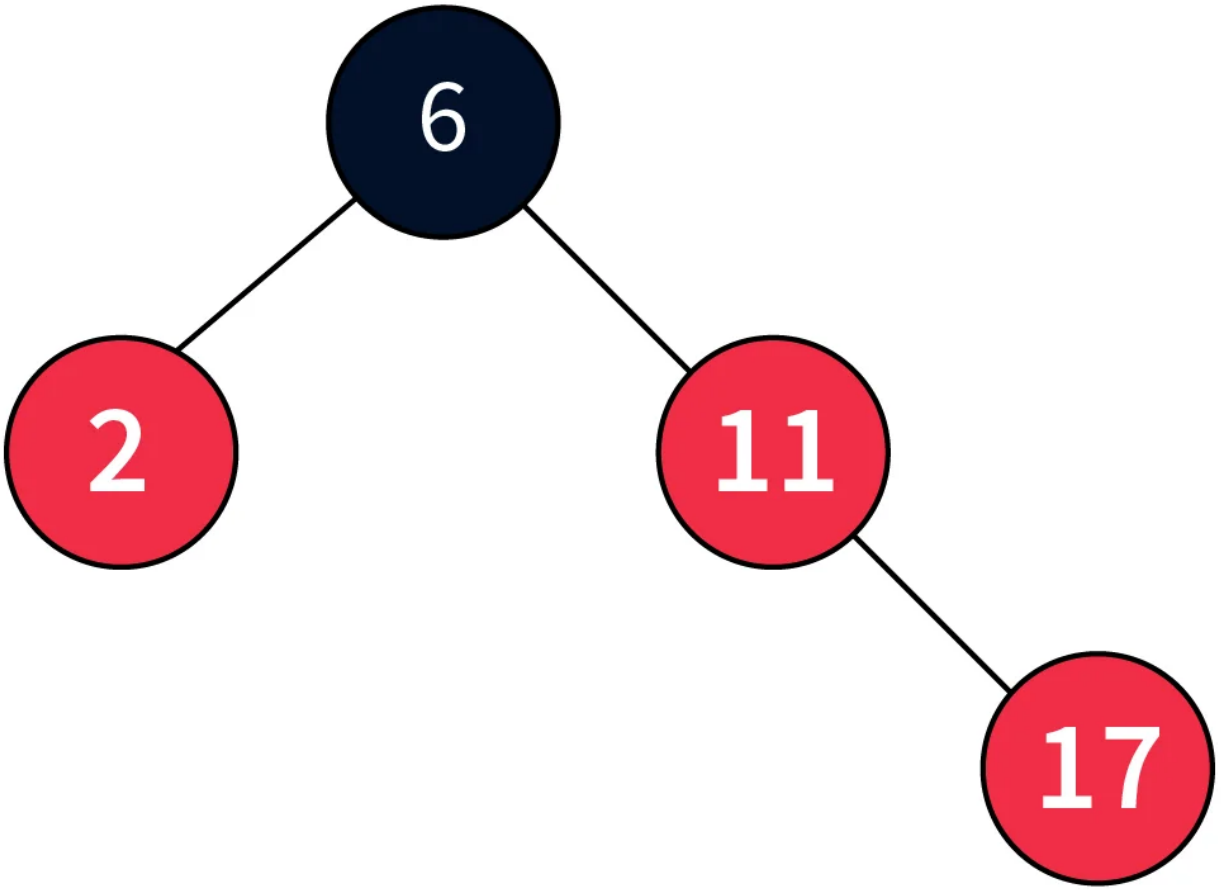
2, 6, 11, 17, 22, 30, 49, 58



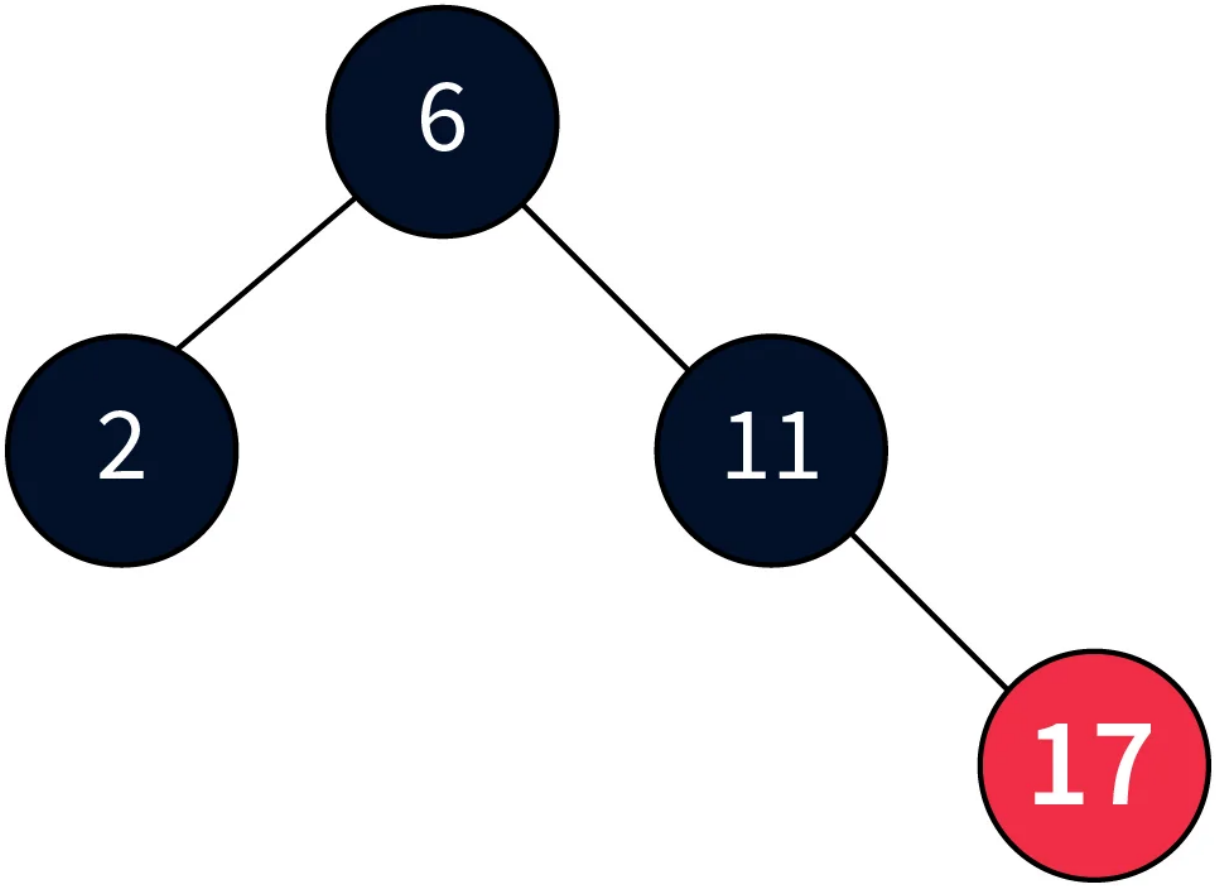
2, 6, 11, 17, 22, 30, 49, 58



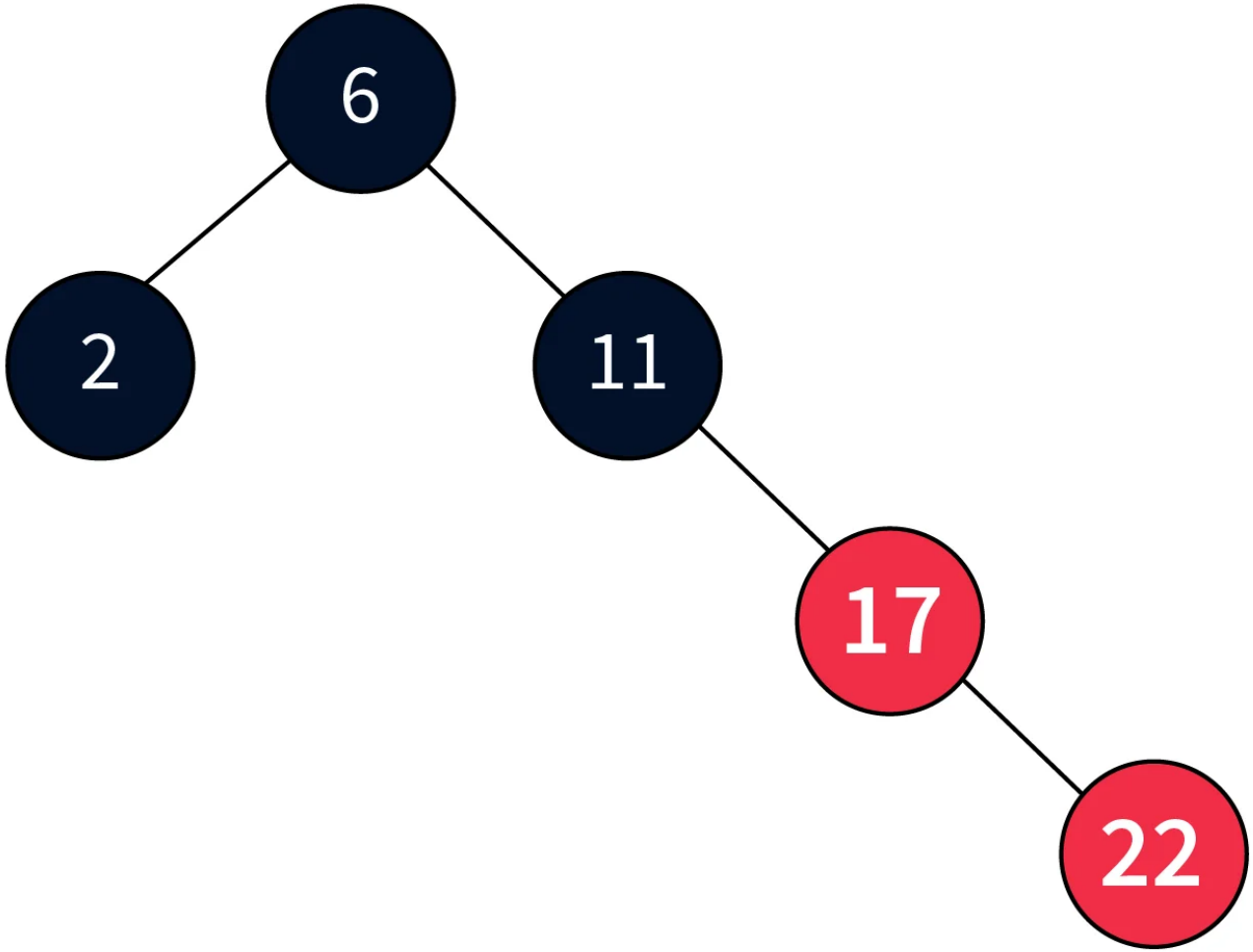
2, 6, 11, 17, 22, 30, 49, 58



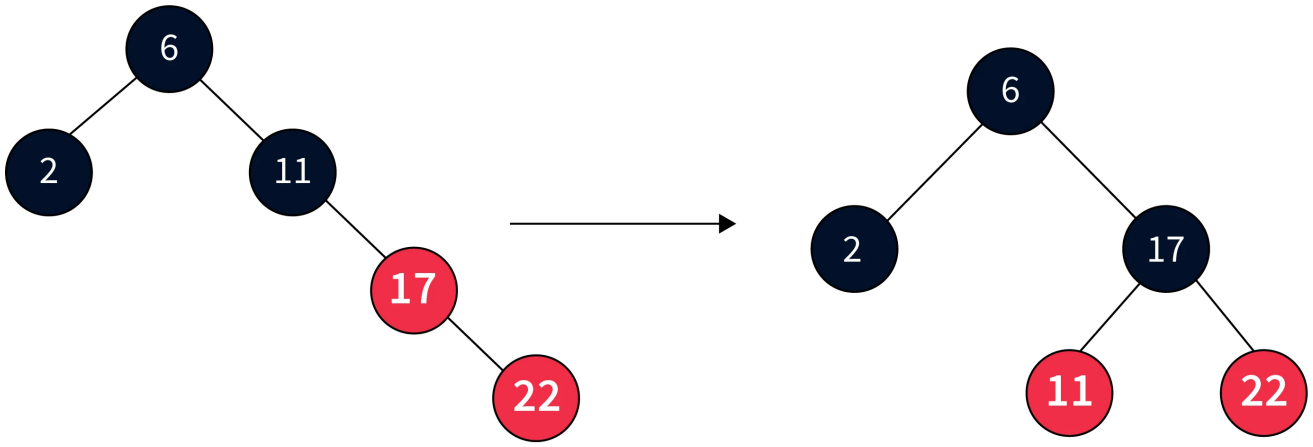
2, 6, 11, 17, 22, 30, 49, 58



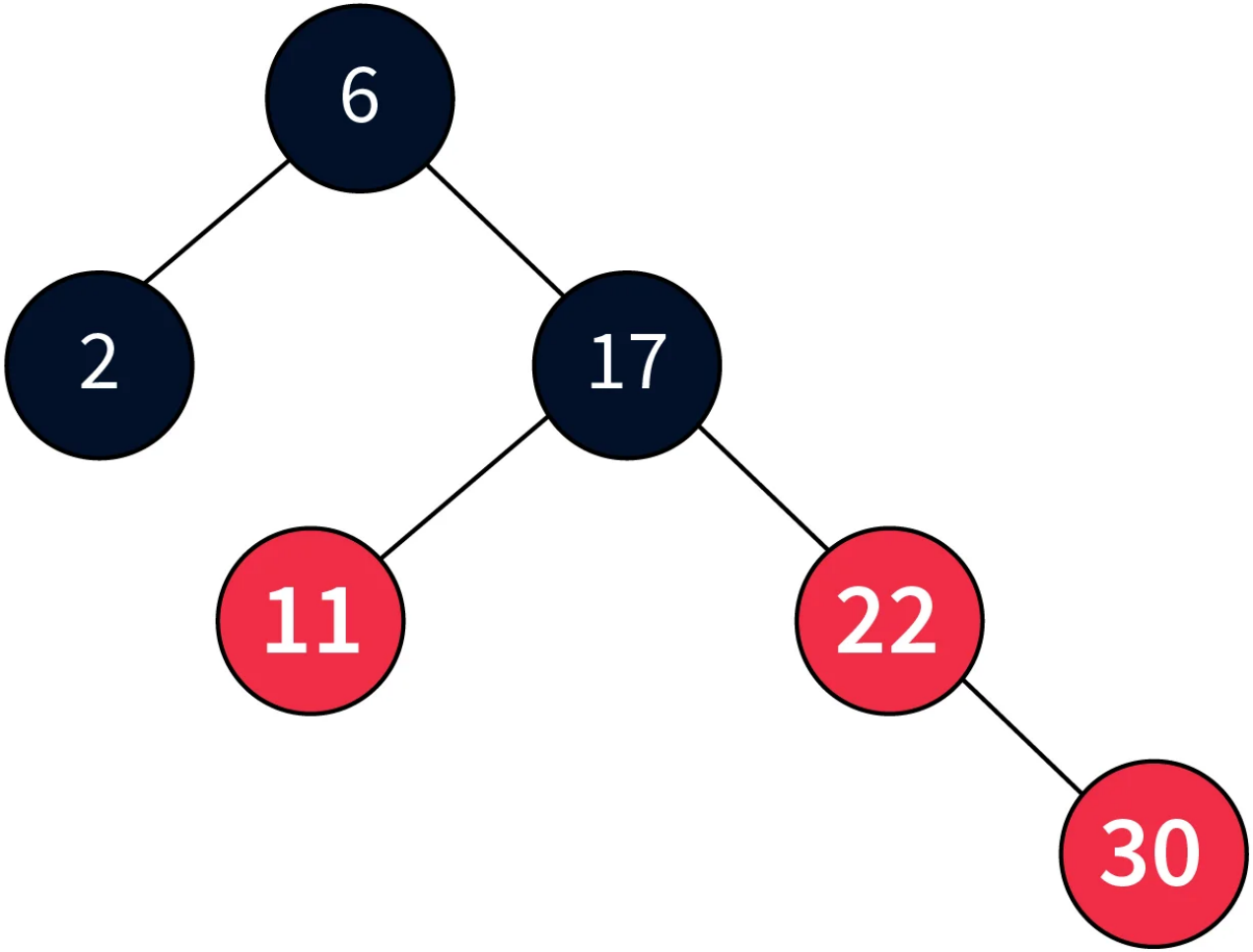
2, 6, 11, 17, 22, 30, 49, 58



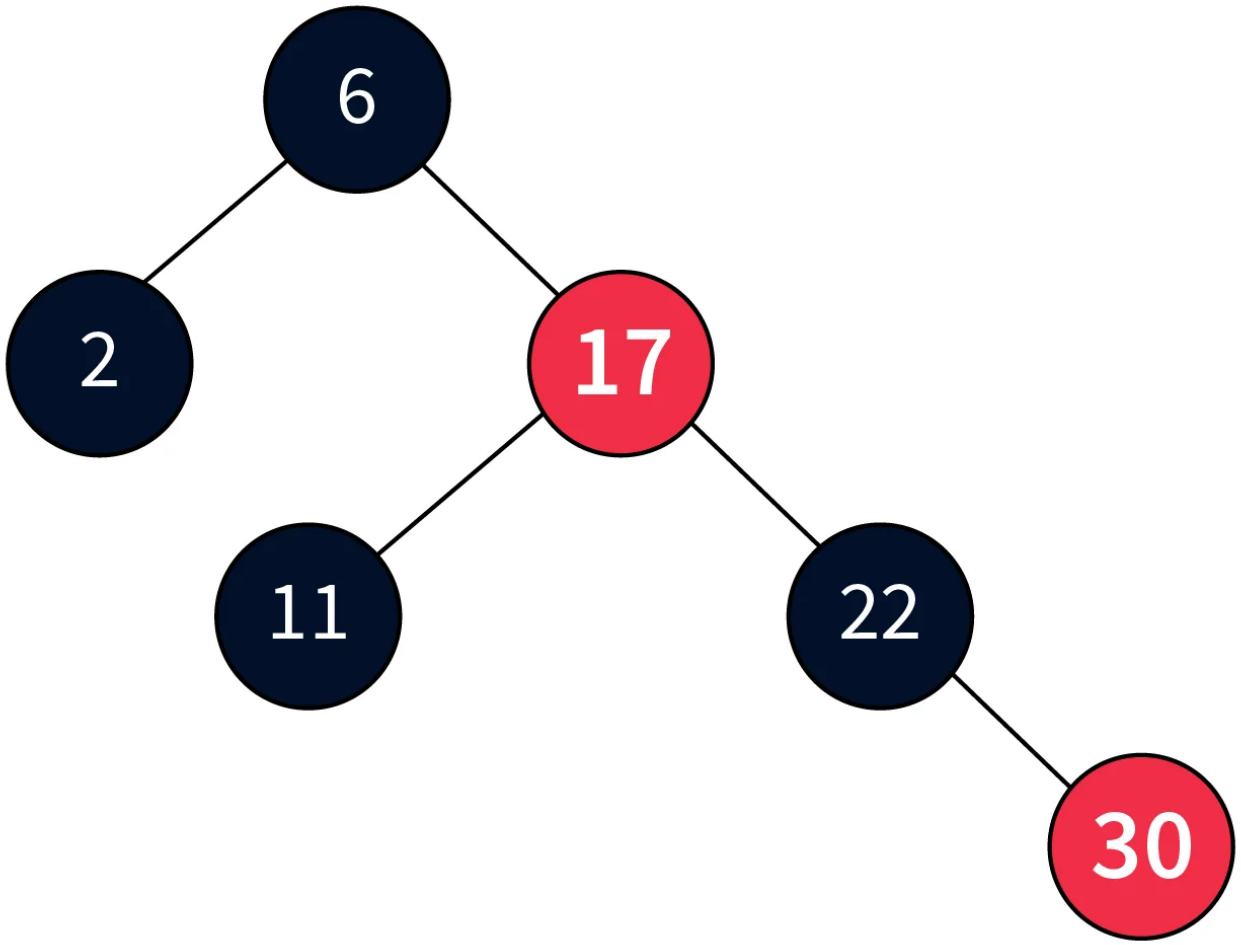
2, 6, 11, 17, 22, 30, 49, 58



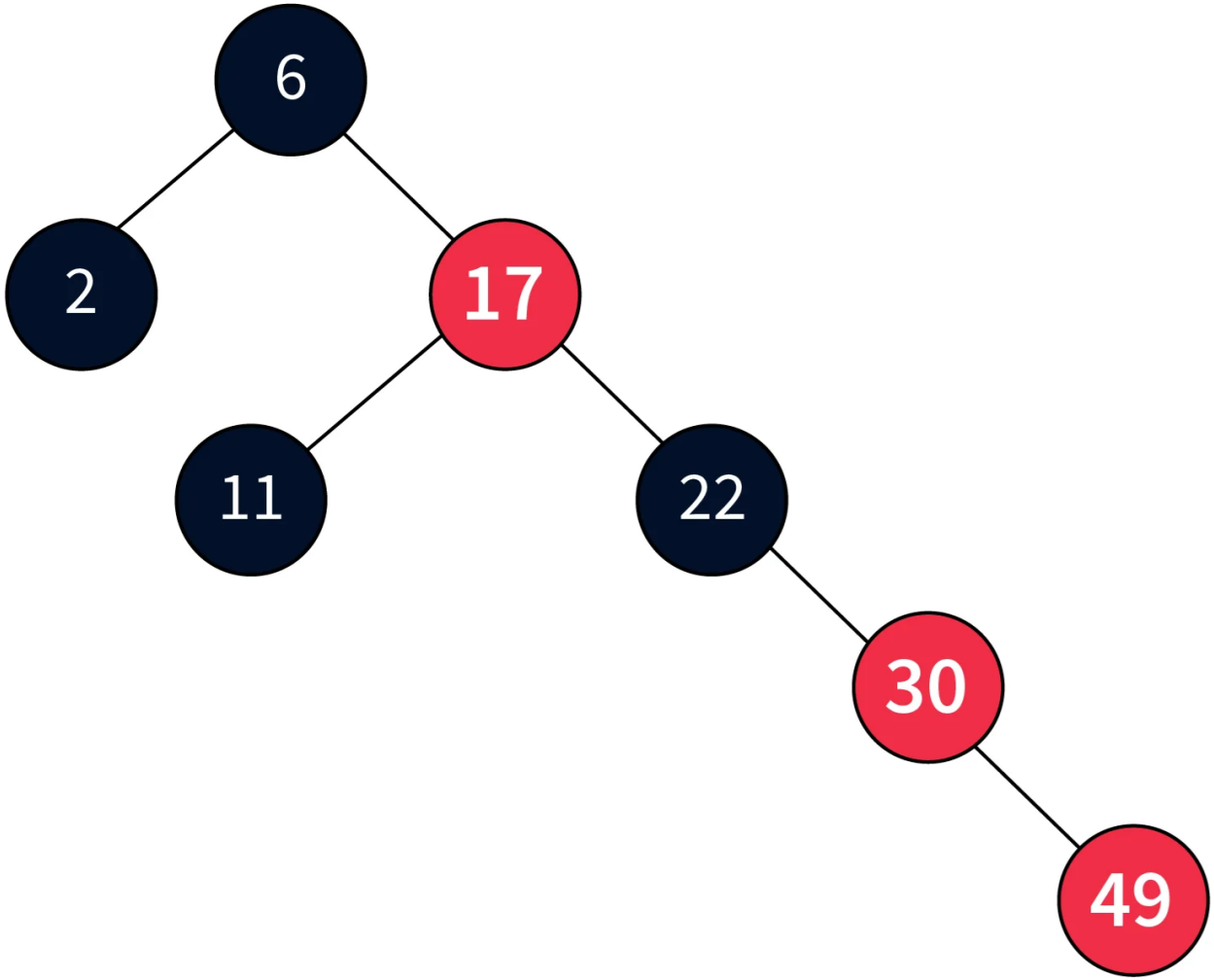
2, 6, 11, 17, 22, 30, 49, 58



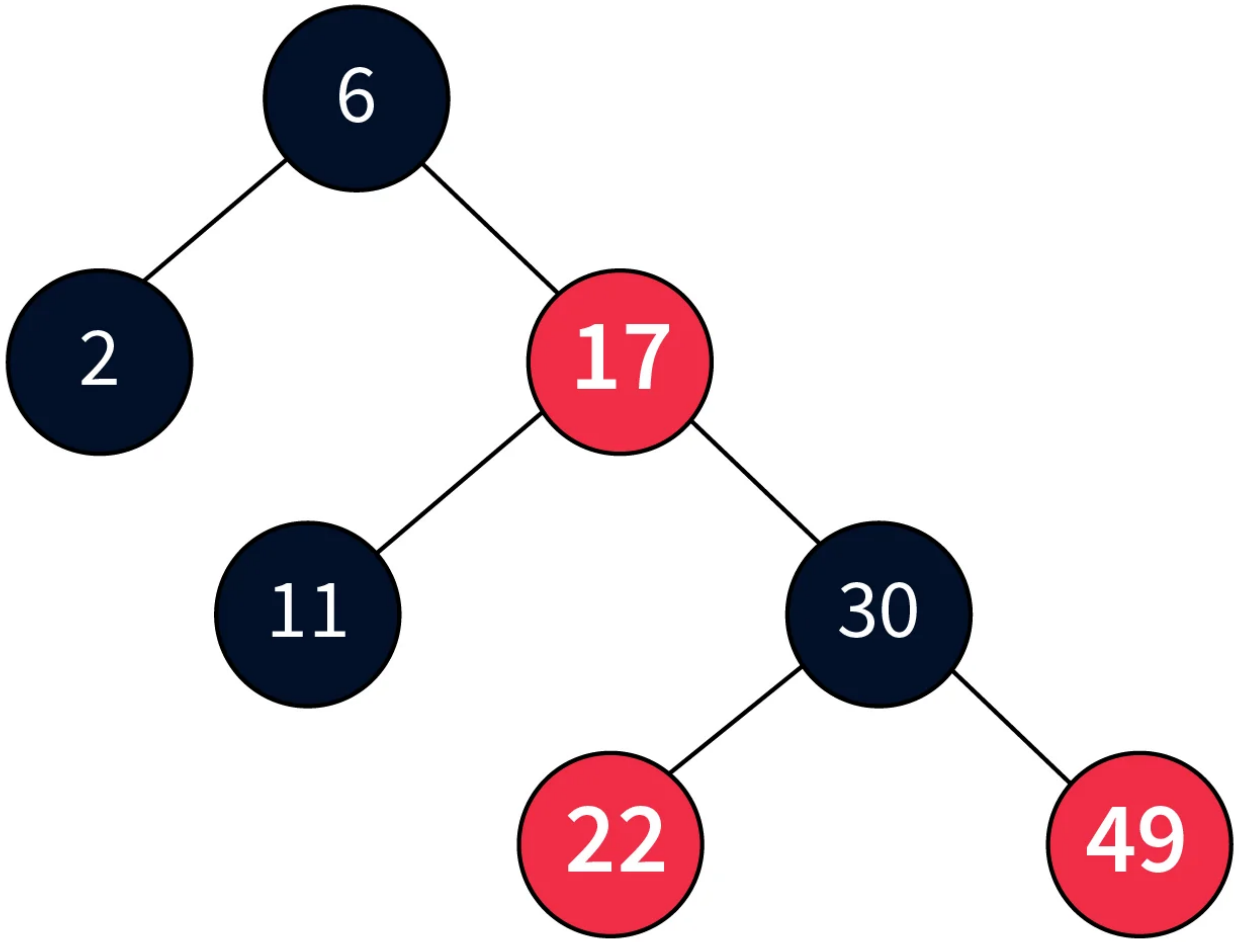
2, 6, 11, 17, 22, 30, 49, 58



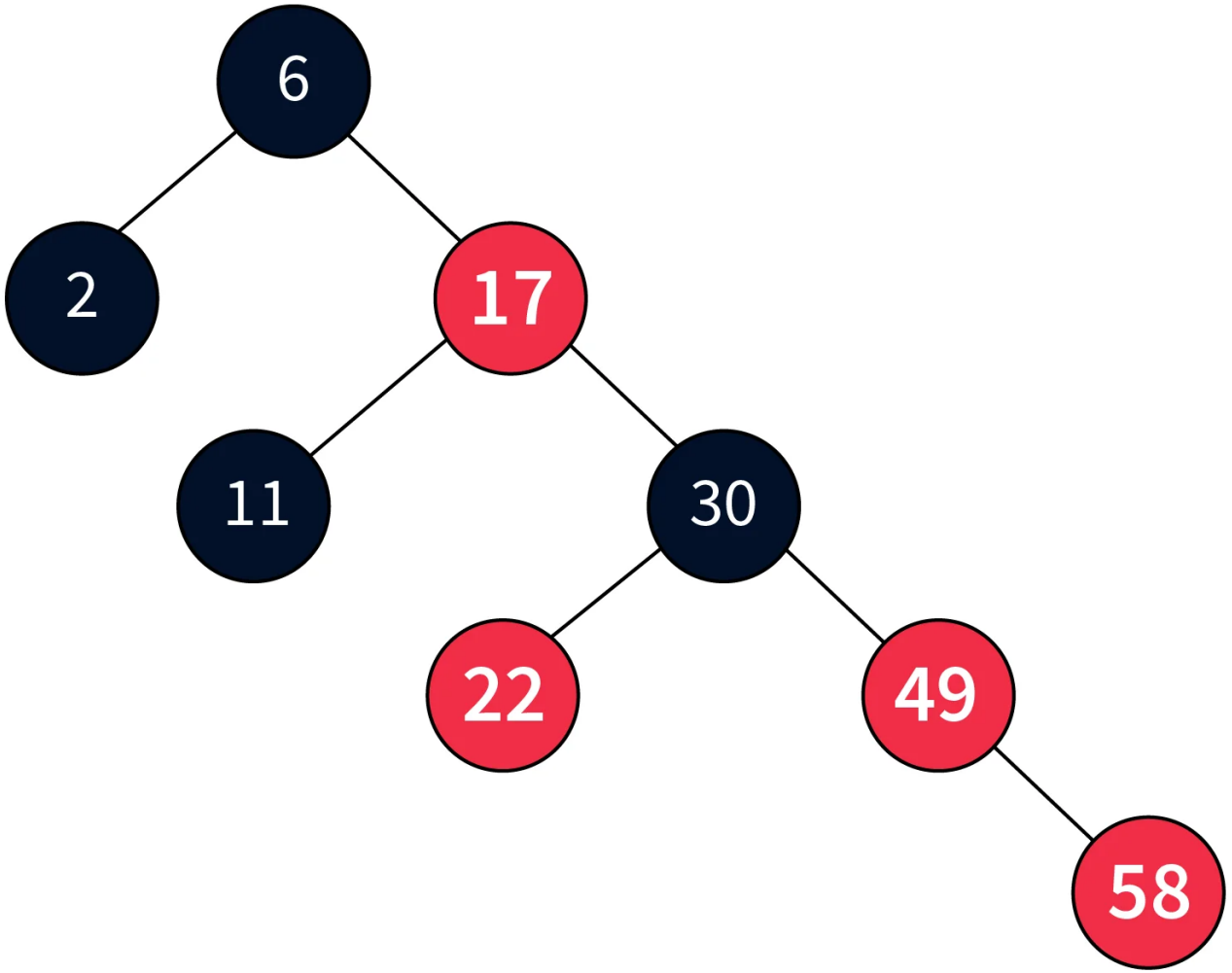
2, 6, 11, 17, 22, 30, 49, 58



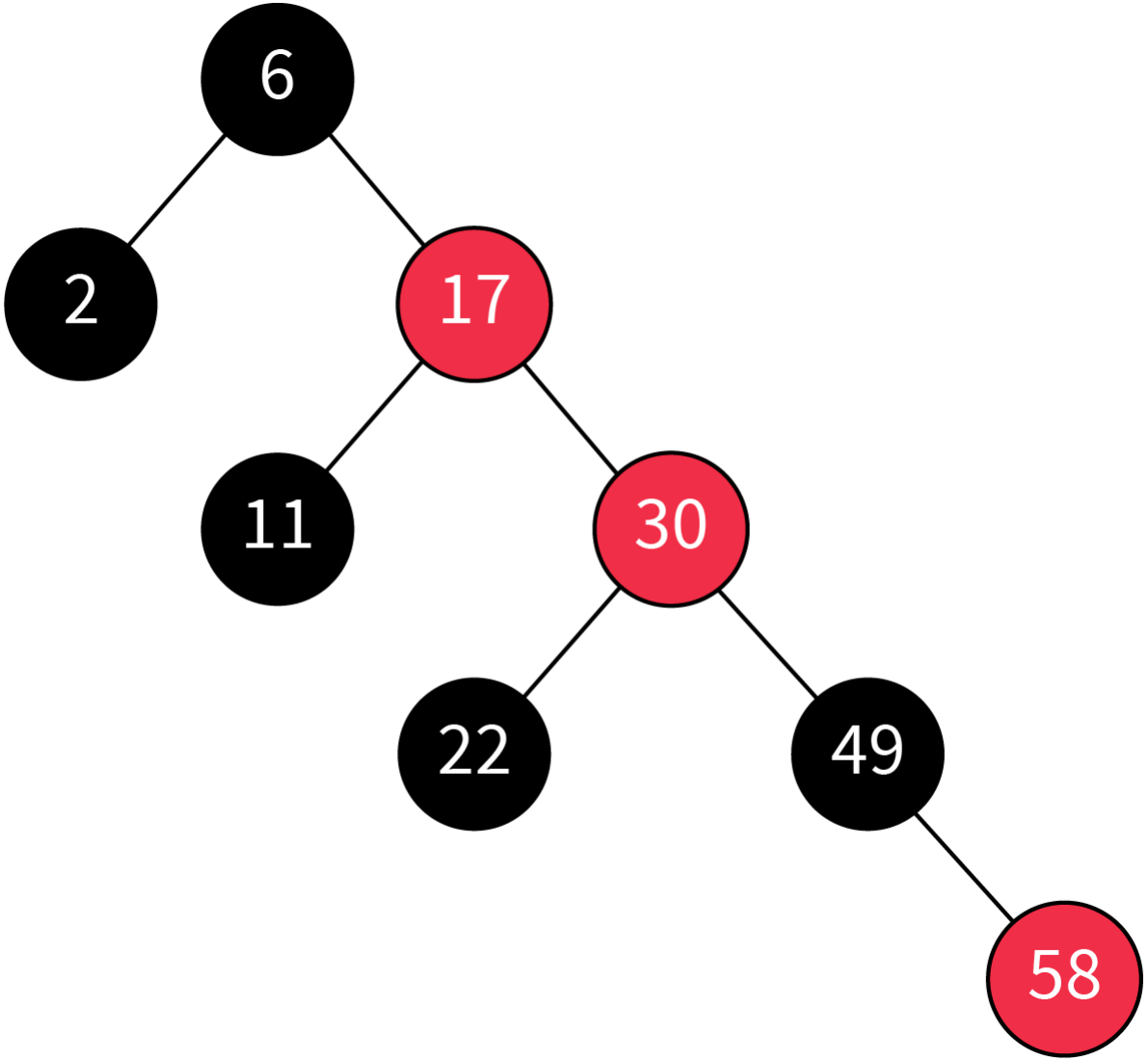
2, 6, 11, 17, 22, 30, 49, 58



2, 6, 11, 17, 22, 30, 49, 58



2, 6, 11, 17, 22, 30, 49, 58



Assignment

1. Insertion: 5,16,22,45,2,10,18,30,50,12,1
2. Deletion in Red Black Tree

Binary Tree through Greedy Strategy

- Binary Search Trees: build the tree based on insertion rules
  - not greedy choices
- Binary Tree using a Greedy Strategy
  - problem: Huffman Coding

Huffman Coding

- data compression: construct a binary tree
  - most frequent characters have the shortest paths to the root, effectively minimizing the total number of bits required to store the data
- Greedy Algorithm: at every step, make the locally optimal choice
  - merging the two smallest frequencies to achieve the globally optimal result
  - a tree with the minimum weighted path length
- Very popular and simple algorithm to construct a prefix free code
  - Optimal prefix code: shortest expected length

Huffman Coding

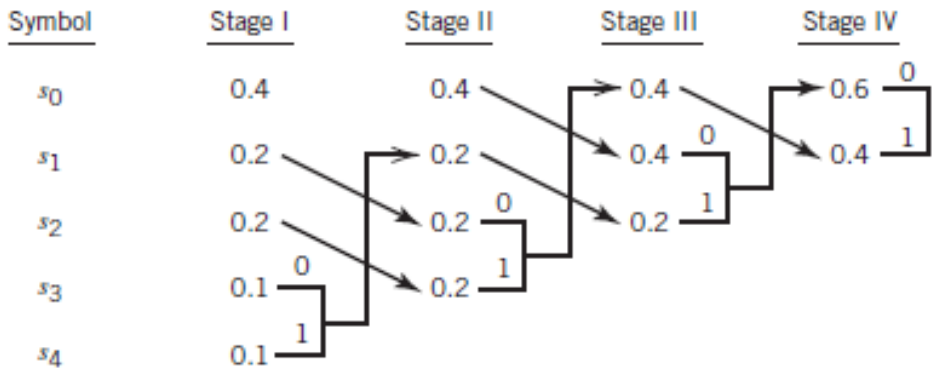
To be specific, the Huffman encoding algorithm

1. The source symbols are listed in order of decreasing probability
   - Two source symbols of lowest probability are assigned 0 and 1: splitting stage
2. These two source symbols are then combined into a new source symbol with a probability equal to the sum of the two original probabilities.
   - The probability of the new symbol is placed in the list in accordance with its value.
3. The procedure is repeated until we are left with a final list of source statistics (symbols) of only two for which the symbols 0 and 1 are assigned

Huffman Code

Symbol source	Probability of occurrences
s_0	0.4
s_1	0.2
s_2	0.2
s_3	0.1
s_4	0.1

Huffman Coding Algorithm



- The code for each symbol is found by backtracking along the tree to each symbol (stages: IV,III,II,I)

Symbol	Probability	Codeword	Length
s_0	0.4	00	2
s_1	0.2	10	2
s_2	0.2	11	2
s_3	0.1	010	3
s_4	0.1	011	3

Note: Lowest probability symbols have codewords of equal length

Encode the given sequence "Ghumne Mechmathi Andho Manche" using Huffman Coding

Symbol	Count
G	1
h	5
u	1
m	2
n	3
e	3
M	2
c	2
a	2
t	1
i	1
A	1
d	1
o	1
Space	3

Encode the given sequence "Ghumne Mechmathi Andho Manche" using Huffman Coding

Symbol	Count	Probability
h	5	5/29
n	3	3/29
e	3	3/29
Space	3	3/29
m	2	2/29
M	2	2/29
c	2	2/29
a	2	2/29
G	1	1/29
u	1	1/29
t	1	1/29
i	1	1/29
A	1	1/29
d	1	1/29
o	1	1/29

Symbol	Probability	Stage 1	S2	S3	S4	S5	S6
h	0.172	0.172	0.172	0.172	0.172	0.172	0.172
n	0.103	0.103	0.103	0.103	0.103	0.136	0.136
e	0.103	0.103	0.103	0.103	0.103	0.103	0.136
space	0.103	0.103	0.103	0.103	0.103	0.103	0.103
m	0.068	0.068	0.068	0.068	0.102	0.103	0.103
M	0.068	0.068	0.068	0.068	0.068	0.102	0.103
c	0.068	0.068	0.068	0.068	0.068	0.068	0.102
a	0.068	0.068	0.068	0.068	0.068	0.068	0.068
G	0.034	0.068	0.068	0.068	0.068	0.068	0.068
u	0.034	0.034	0.068	0.068	0.068	0.068	
t	0.034	0.034	0.034	0.068	0.068		
i	0.034	0.034	0.034	0.034			
A	0.034	0.034	0.034				
d	0.034	0.034					
o	0.034						

S6	S7	S8	S9	S10	S11	S12	S13
0.172	0.172	0.205	0.206	0.272	0.308	0.411	0.58
0.136	0.136	0.172	0.205	0.206	0.272	0.308	0.411
0.136	0.136	0.136	0.172	0.205	0.206	0.272	
0.103	0.136	0.136	0.136	0.172	0.205		
0.103	0.103	0.136	0.136	0.136			
0.103	0.103	0.103	0.136				
0.102	0.103	0.103					
0.068	0.102						
0.068							

Symbol	Count	Probability (p_k)	Codeword	Length (l_k)	$p_k l_k$	$-p_k \log_2(p_k)$
h	5	$5/29$	000	3	0.5172	0.43725
n	3	$3/29$	100	3	0.3103	0.33858
e	3	$3/29$	101	3	0.3103	0.33858
Space	3	$3/29$	110	3	0.3103	0.33858
m	2	$2/29$	0101	4	0.2758	0.26606
M	2	$2/29$	0110	4	0.2758	0.26606
c	2	$2/29$	0111	4	0.2758	0.26606
a	2	$2/29$	1110	4	0.2758	0.26606
G	1	$1/29$	1111	4	0.1379	0.16751
u	1	$1/29$	00100	5	0.1724	0.16751
t	1	$1/29$	00101	5	0.1724	0.16751
i	1	$1/29$	00110	5	0.1724	0.16751
A	1	$1/29$	00111	5	0.1724	0.16751
d	1	$1/29$	01000	5	0.1724	0.16751
o	1	$1/29$	01001	5	0.1724	0.16751
					3.7236	3.6898

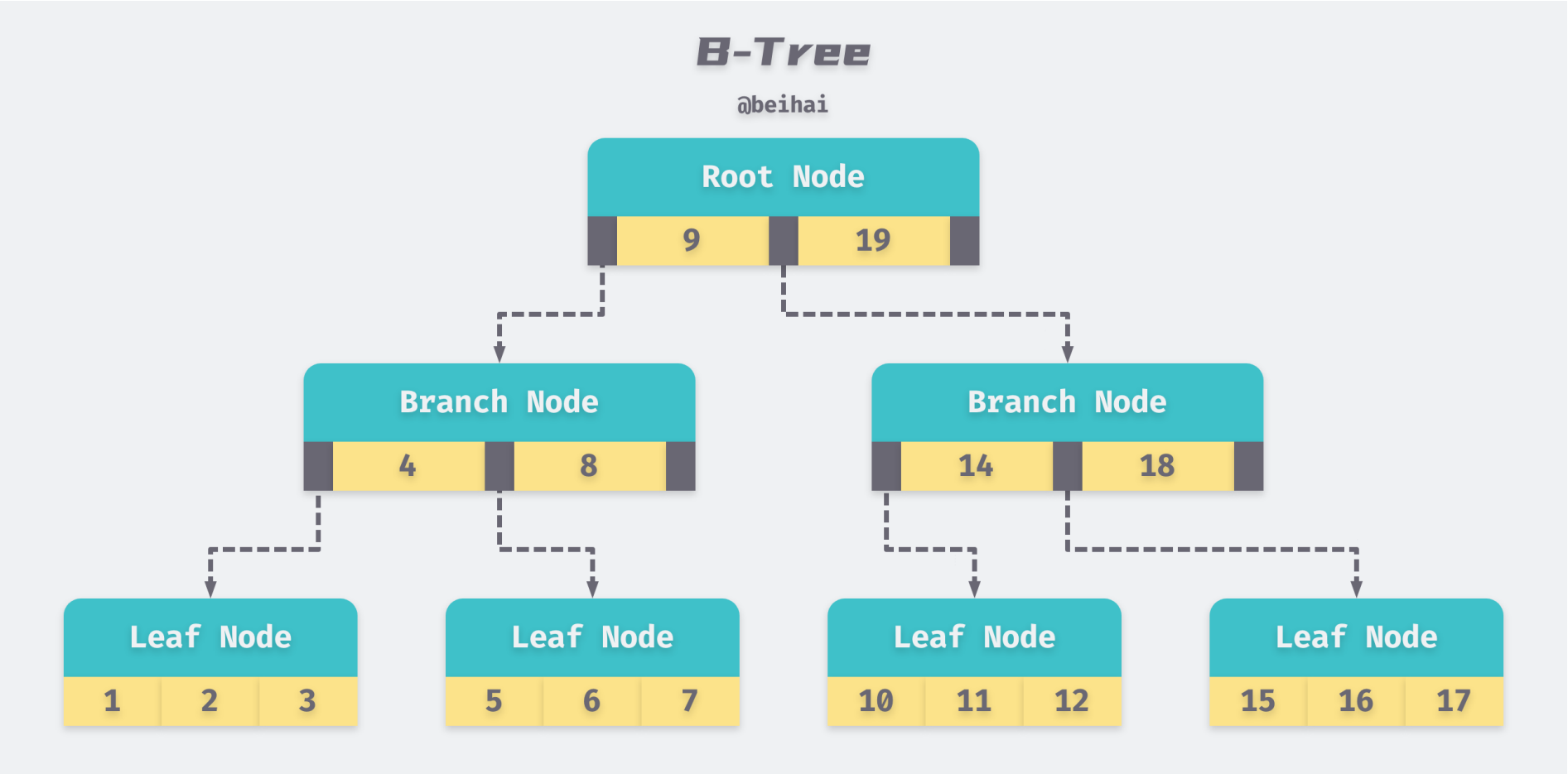
B-Tree

- self-balancing multi-way ($m-$way) search tree
  - each node can have more than two children
  - unlike binary trees
- keeps data sorted
- allows operations in $O(\log n)$ time
- works well for large data stored on disk

Properties of B-Tree

For a B-tree of **order** m:

- Each node can have at most m children
- Each node has at least $\lceil \frac{m}{2} \rceil - 1$ children except
  - root: 2
  - leaf: 0
- A node with m children has $m - 1$ keys
- All leaf nodes are at the same level
- Keys in a node are sorted

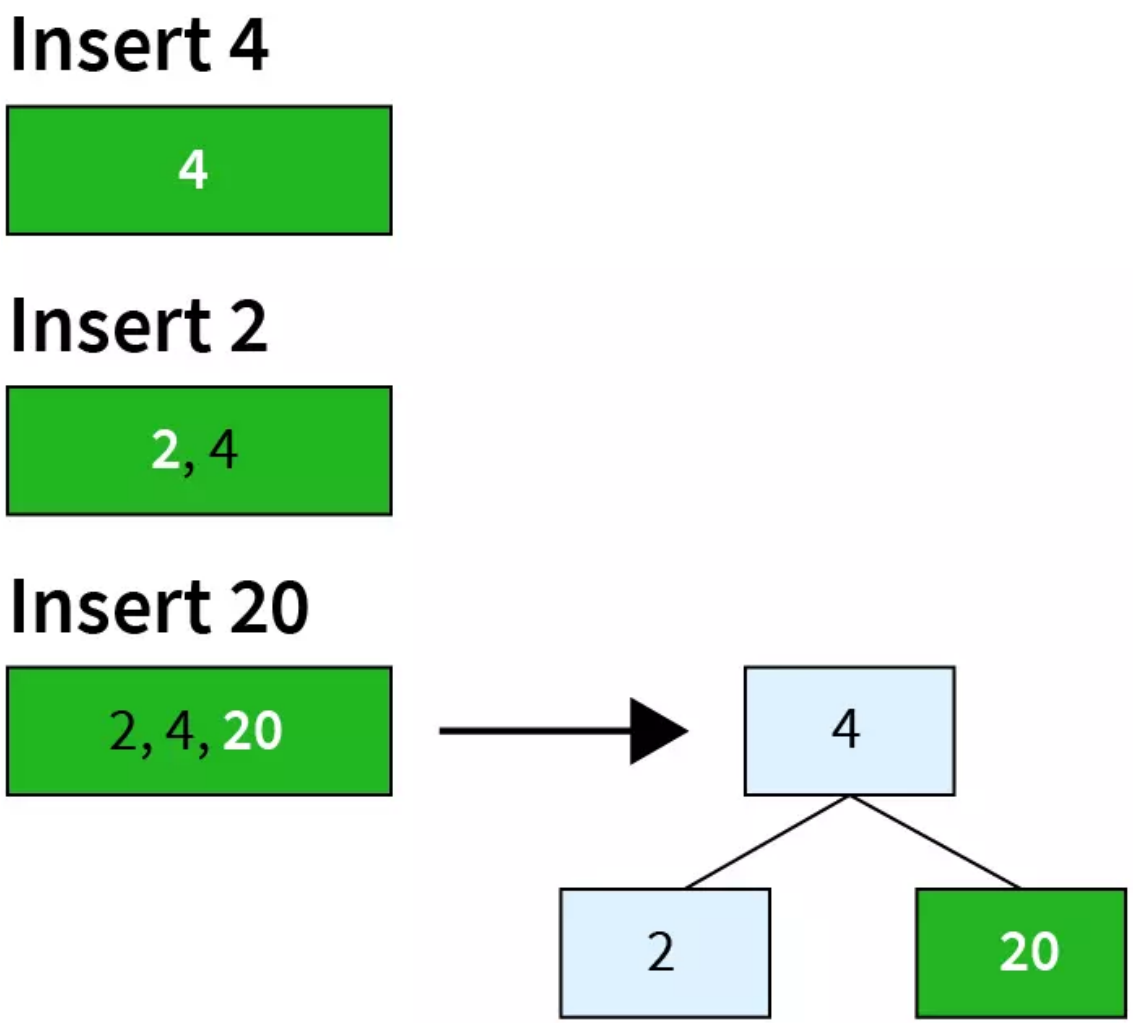

B-Tree
@beihai
Root Node
9
19
Branch Node
4
8
Branch Node
14
18
Leaf Node
1
2
3
Leaf Node
5
6
7
Leaf Node
10
11
12
Leaf Node
15
16
17

Insertion

1. Calculate the maximum number of keys in the node based on the order of the B tree
2. If the tree is empty, a root node is allocated and the key is inserted and acts as the root node
3. Search the appropriate node for insertion
4. If the node is full:
   - 4.1 Insert the elements in increasing order
   - 4.2 If the elements are greater than the maximum number of keys, split at the median
   - 4.3 Push the median key upwards and split the left and right keys as left and right child respectively
5. If the node is not full, insert the node in increasing order

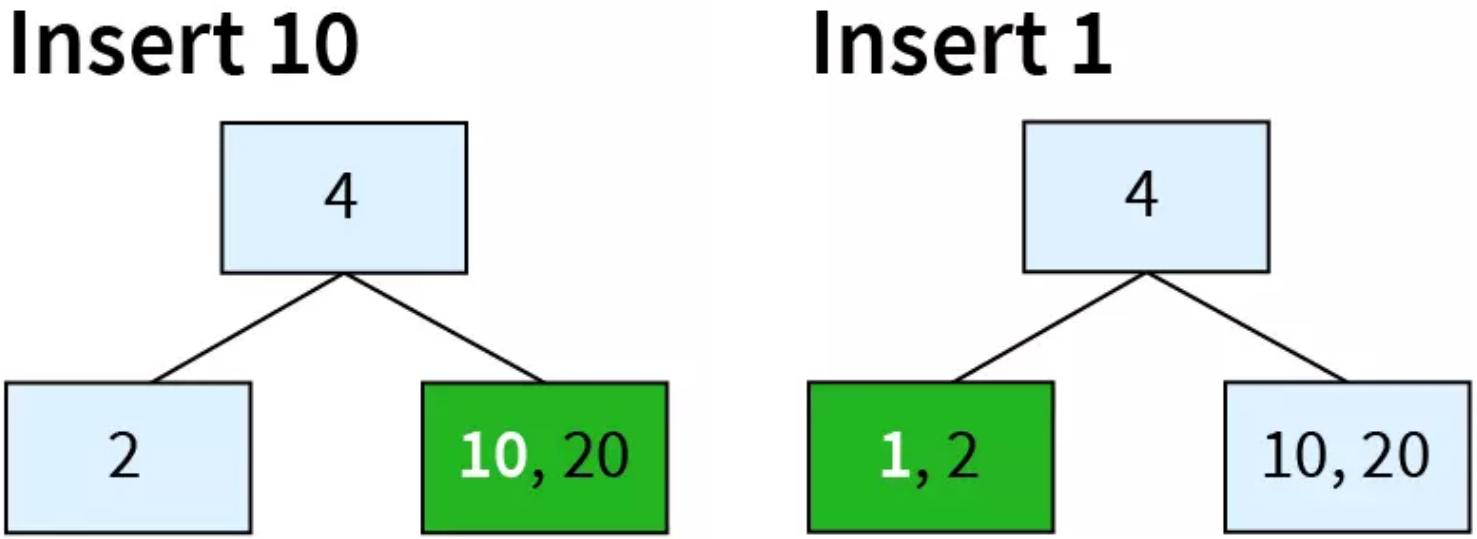
create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

Since $m = 3$, max number of keys for a node $= m - 1 = 2$



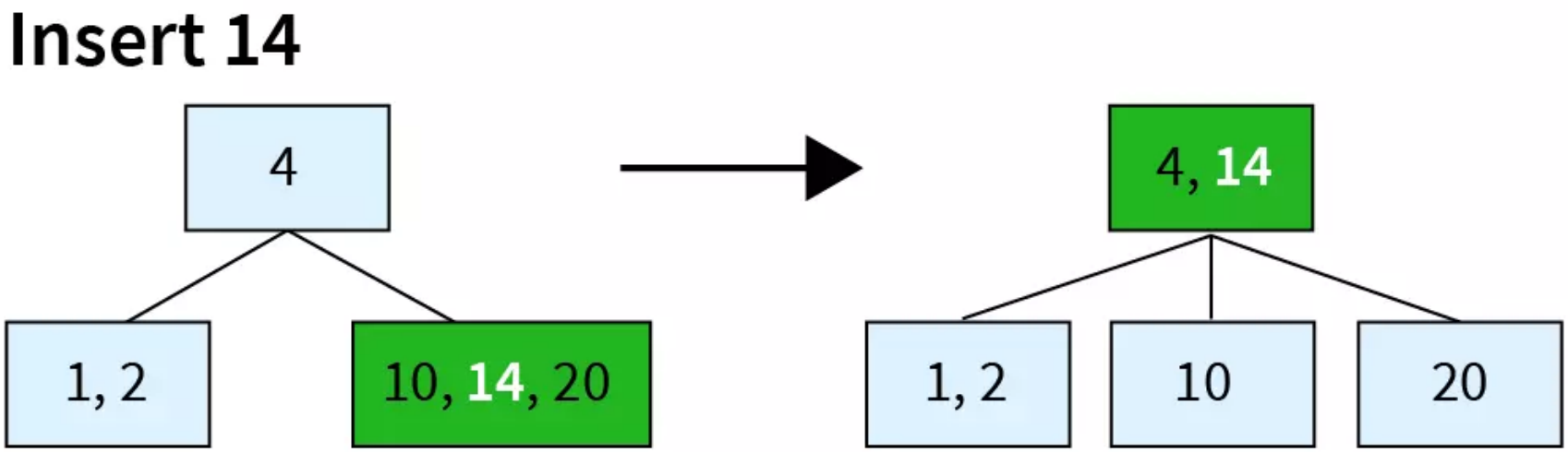
create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

Since $m = 3$, max number of keys for a node $= m - 1 = 2$



create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

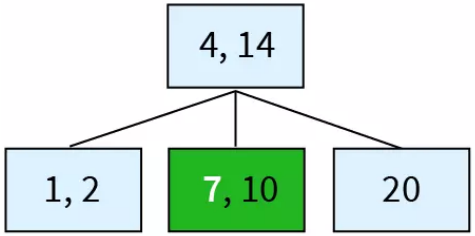
Since $m = 3$, max number of keys for a node $= m - 1 = 2$



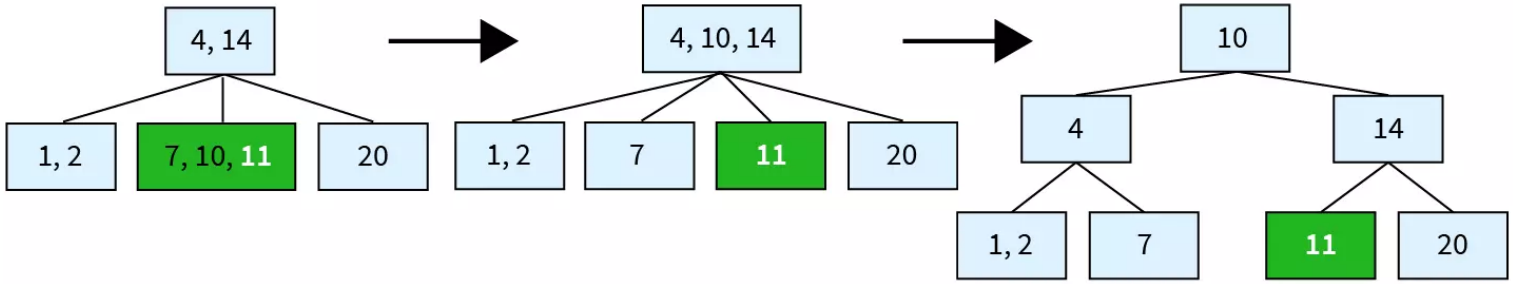
create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

Since $m = 3$, max number of keys for a node $= m - 1 = 2$

Insert 7

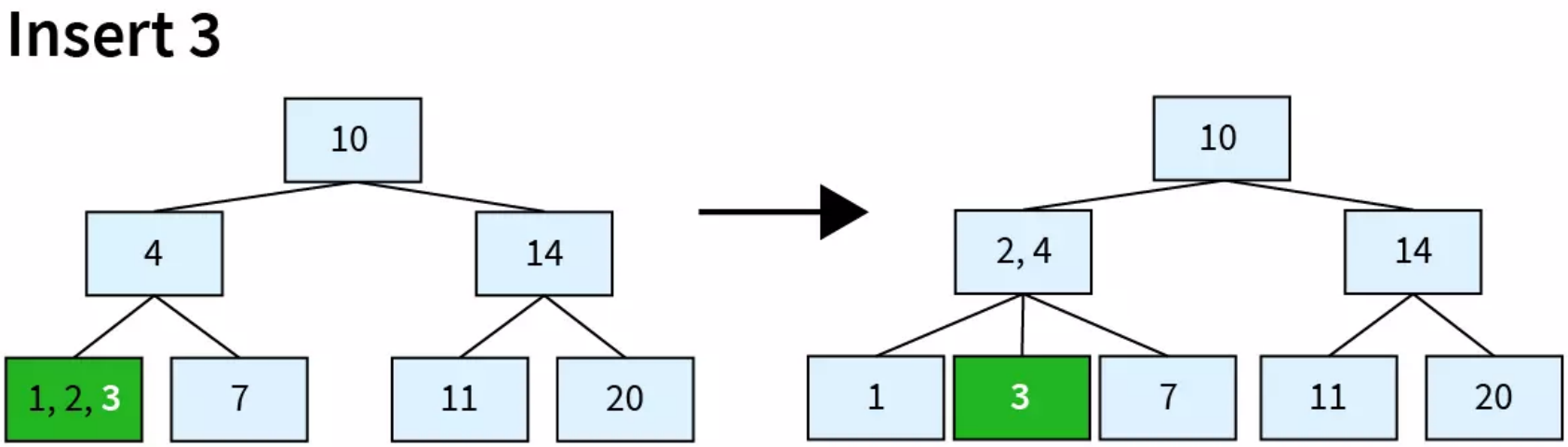


Insert 11



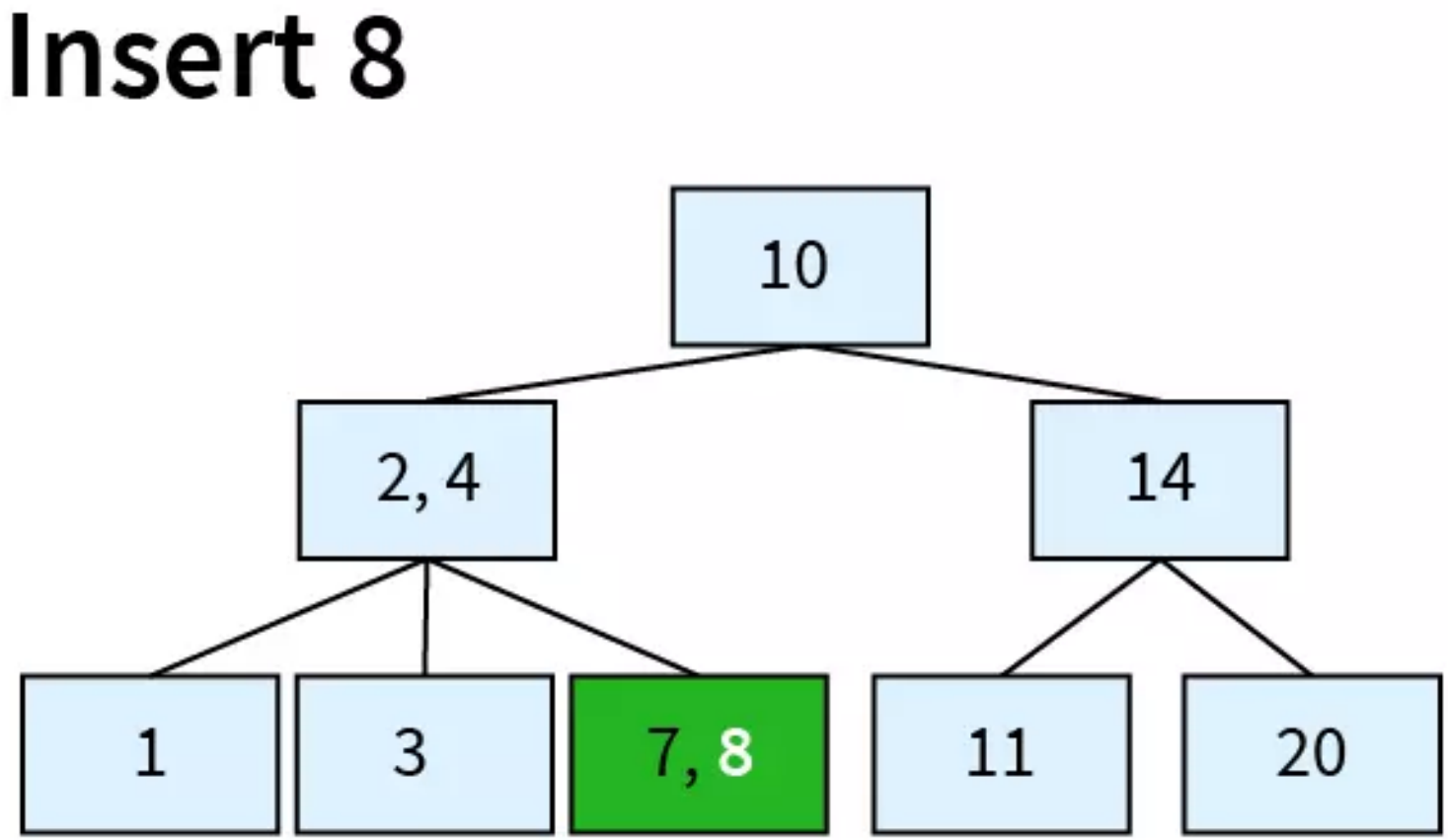
create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

Since $m = 3$, max number of keys for a node $= m - 1 = 2$



create a B tree of order 3 where the elements to be inserted are 4, 2, 20, 10, 1, 14, 7, 11, 3, 8

Since $m = 3$, max number of keys for a node $= m - 1 = 2$



When M= even

- Max Children per node $M = 4$ children
- median is not unique
- When splitting a node with 4 keys (e.g., keys K_1, K_2, K_3, K_4):
  - median key: 2nd or 3rd key, depending on implementation
  - promoted up to the parent node
  - remaining keys split into two separate nodes (Left and Right)
- Left bias: Promoted the left key (K_2) to move up to the parent
  - Left Node Keys: $[K_1]$
  - Right Node Keys: $[K_3, K_4]$
- Right bias: Promoted the right key (K_3) to move up to the parent
  - Left Node Keys: $[K_1, K_2]$
  - Right Node Keys: $[K_4]$

Assignment

1. Deletion
2. B+ Tree